

Multi-Turn Distributed Prompt Injection Detection

COMP4531 Deep Learning — Final Project Report

Rock Lambros
kyriakos.lambros@du.edu

April 2026

Platform: NVIDIA Jetson Orin AGX (64GB RAM, 2048-core Ampere GPU)

Abstract

Single-turn prompt injection classifiers achieve high accuracy on known attack distributions, yet they fail against multi-turn attacks that distribute malicious intent across several conversation turns. Each turn in isolation appears benign; the adversarial signal emerges only from how turns relate over time. We present a dual-encoder architecture that pairs a frozen single-turn GRU encoder (2.6M parameters) with a trainable sequence-level LSTM (27K parameters) to carry forward accumulated context across turns. On 73,390 single-turn samples from eight public datasets, the Chollet heuristic correctly predicts that TF-IDF bag-of-bigrams ($F1 = 0.834$) outperforms all deep learning models including transformers at this data scale. On 7,000 synthetic multi-turn conversations modeled after published attack strategies (Crescendo, Foot-in-the-Door, fragment distribution, instruction layering), the temporal architecture achieves $F1 = 0.989$, a +10.3 percentage point improvement over the best per-turn classifier ($F1 = 0.887$). Adding attention raises $F1$ to 0.992 while providing interpretability, and threshold tuning reaches 0.995. These results confirm that temporal modeling captures escalation patterns invisible to single-turn classification.

Contents

1	Problem Statement and Motivation	3
1.1	The Problem	3
1.2	Why Deep Learning	3
1.3	Contribution	3
2	Data and Preprocessing	3
2.1	Single-Turn Datasets	3
2.2	Data Bias Analysis	4
2.3	Cleaning Pipeline	4
2.4	Synthetic Multi-Turn Data	4
2.5	Tokenization	4
2.6	I/O Variable Specification	5
2.7	Chollet Heuristic Analysis	5
3	Metric Justification	6

4	Model Architecture and Iteration Plan	6
4.1	Iteration 0: Baselines	6
4.2	Iterations 1–4: Single-Turn Deep Learning Models	6
4.3	Iterations 4b–4c: Transformer Comparison	7
4.4	Iteration 5: Multi-Turn Classifier	7
4.5	Iteration 6: Attention	7
4.6	Iteration 7: Threshold Tuning	7
5	Results	8
5.1	Single-Turn Results	8
5.2	Baseline Error Analysis	9
5.3	Multi-Turn Results (Core Finding)	10
5.4	Multi-Turn Example Analysis	10
5.5	Attention Interpretability	12
5.6	Threshold Analysis	13
5.7	Statistical Significance Discussion	14
6	Analytical Analysis	15
6.1	Overfitting and Underfitting Detection	15
6.2	Convergence Comparison	16
6.3	AUC Comparison	17
6.4	Error Pattern Analysis	17
7	Discussion	18
7.1	Why Multi-Turn Works	18
7.2	Iterative Refinement Summary	18
7.3	Limitations	19
7.4	Future Work	19
8	Reproducibility	19
	References	19

1 Problem Statement and Motivation

1.1 The Problem

Single-turn prompt injection detection achieves high accuracy on known attack patterns. ProtectAI’s DeBERTa model reaches 99%+ F1 on published benchmarks [1]. This performance is limited to known attack distributions. Novel attacks, adversarially crafted inputs, and multi-turn strategies routinely evade these detectors [2, 3]. Real-world attacks against AI agent systems increasingly distribute malicious intent across multiple conversation turns, where each individual turn appears benign in isolation.

This is the **Crescendo attack pattern** [4]: an attacker gradually escalates across turns, establishing trust and context before making the exploitative request. The **Foot-in-the-Door technique** [5] similarly leverages compliance momentum across turns. Vassilev [3] extends Gödel’s incompleteness theorem to argue that no single-turn classifier can theoretically detect all such attacks.

1.2 Why Deep Learning

The multi-turn attack signal is fundamentally temporal: earlier turns create context that later turns exploit. LSTM and GRU architectures are designed to model exactly this type of sequential dependency through their gating mechanisms. A bag-of-words or per-turn classifier has no mechanism to carry forward accumulated risk state.

1.3 Contribution

This project builds a multi-turn distributed prompt injection detection system using a dual-encoder architecture: a frozen single-turn encoder paired with a sequence-level LSTM that carries forward context across turns. We demonstrate a **+10.3 percentage point F1 improvement** over per-turn classification on multi-turn attack data (F1 = 0.989 vs. 0.887).

2 Data and Preprocessing

2.1 Single-Turn Datasets

Eight HuggingFace datasets were merged (expanded from the original three to enable transformer comparison):

Table 1: Source datasets with sample counts and types.

Dataset	Samples	Type
deepset/prompt-injections	662	Binary classification
xTRam1/safe-guard-prompt-injection	10,296	Binary classification
neuralchemy/Prompt-injection-dataset	6,274	Binary classification
imoxto/prompt_injection_cleaned_dataset-v2	40,000	Binary w/ system prompt
reshabhs/SPML_Chatbot_Prompt_Injection	16,012	Gandalf CTF attacks
TrustAIRLab/in-the-wild-jailbreak (jailbreak)	1,405	Real-world jailbreaks
TrustAIRLab/in-the-wild-jailbreak (regular)	13,735	Real-world benign
jackhhao/jailbreak-classification	1,306	Curated binary

After cleaning (deduplication, whitespace normalization, length filtering), 73,390 samples remained, split 70/15/15 stratified on label: 51,373 train / 11,008 val / 11,009 test. Class balance: 64% benign / 36% injection.

2.2 Data Bias Analysis

Three sources of bias require discussion.

Source distribution dominance. The `imoxto` dataset contributes 40,000 of 73,390 samples (54.5% of training data). This dataset contains system prompt context that other sources lack, potentially biasing models toward detecting injection patterns co-occurring with system prompts rather than injection content alone. The Gandalf CTF attacks (16,012 samples, 21.8%) skew toward game-style jailbreaks that may not represent production attack distributions.

Text length correlation with class. Injection texts tend to be longer than benign queries (mean 112 tokens vs. 63 tokens for benign). Models could partially exploit length as a proxy feature rather than learning semantic content. The TF-IDF baselines are particularly susceptible to this, as longer documents produce denser feature vectors.

Class imbalance handling. The 64/36 benign/injection split is moderate. We chose *not* to apply class weights or oversampling for two reasons: (1) the imbalance ratio (1.78:1) is mild enough that standard loss functions converge without correction, and (2) applying class weights in preliminary experiments increased recall by 2 points at the cost of 5 points of precision, a poor trade-off for a security classifier where false positives erode user trust. Stratified splitting preserves the natural distribution in all partitions.

2.3 Cleaning Pipeline

Nine cleaning steps applied in order: column normalization, label normalization (0 = benign, 1 = injection), whitespace stripping, internal whitespace collapse, exact deduplication, near-deduplication (lowercase + strip punctuation), short text removal (<3 tokens), long text removal (>2048 chars), and logging.

Removals: 167 exact duplicates, 40 near-duplicates, 132 empty/short, 489 too-long.

2.4 Synthetic Multi-Turn Data

No public dataset of multi-turn distributed attacks exists. We generated 7,000 synthetic conversations (5,000 train, 1,000 val, 1,000 test) using four strategies:

- **Fragment distribution (40%):** Split injection text into 3–5 fragments interleaved with benign filler.
- **Gradual escalation (30%):** Crescendo pattern; each turn adds specificity toward the goal.
- **Context priming (20%):** Establish persona/authority in early turns, exploit later.
- **Instruction layering (10%):** Each turn adds one constraint, cumulatively overriding safety.

Balanced 50/50 classes, 3–10 turns per conversation. Benign conversations sampled from a pool of 500+ unique filler turns.

2.5 Tokenization

Custom vocabulary of 20,000 tokens built from training data only. Max sequence length: 256 tokens. OOV rate: 0.87% on training, 1.19% on validation.

2.6 I/O Variable Specification

Table 2 specifies all input and output variables for the single-turn models. Table 3 specifies variables for the multi-turn dual-encoder.

Table 2: I/O variable specification for single-turn models (Iterations 0–4).

Variable	Type	Shape	Range	Encoding
<i>Inputs</i>				
Token IDs	int64	$(B, 256)$	$[0, 19999]$	Integer index into 20K vocabulary; 0 = PAD
Sequence lengths	int64	$(B,)$	$[1, 256]$	Actual token count before padding
GloVe embeddings (Iter 2)	float32	$(20000, 100)$	$[-3.0, 3.0]$	Pretrained 100d vectors, frozen
TF-IDF features (Iter 0)	float64	$(B, 10000)$	$[0.0, 1.0]$	Sparse bigram TF-IDF, L2-normalized
<i>Outputs</i>				
Logit	float32	$(B, 1)$	$(-\infty, +\infty)$	Raw BCEWithLogitsLoss input
Probability	float32	$(B, 1)$	$[0.0, 1.0]$	Sigmoid of logit
Predicted label	int	$(B,)$	$\{0, 1\}$	Threshold at 0.5; 0 = benign, 1 = injection

Table 3: I/O variable specification for multi-turn dual-encoder (Iterations 5–7).

Variable	Type	Shape	Range	Encoding
<i>Inputs</i>				
Turn token IDs	int64	$(B, T, 256)$	$[0, 19999]$	Per-turn token sequences; $T \leq 10$ turns
Turn lengths	int64	(B, T)	$[1, 256]$	Per-turn actual lengths
Conversation lengths	int64	$(B,)$	$[3, 10]$	Number of turns per conversation
<i>Intermediate</i>				
Turn embeddings	float32	$(B, T, 32)$	$\mathbb{R}^{32}$	Frozen GRU encoder final hidden state
Attention weights (Iter 6–7)	float32	(B, T)	$[0.0, 1.0]$	Softmax over additive attention scores
<i>Outputs</i>				
Logit	float32	$(B, 1)$	$(-\infty, +\infty)$	Raw BCEWithLogitsLoss input
Probability	float32	$(B, 1)$	$[0.0, 1.0]$	Sigmoid of logit
Predicted label	int	$(B,)$	$\{0, 1\}$	Threshold at 0.5 (or 0.64 after tuning)

2.7 Chollet Heuristic Analysis

Following Chollet [6], we compute the ratio of training samples to mean words per sample to predict which model family will perform best:

- Training samples: 51,373

- Mean words per sample: 87.3
- Ratio: $51,373/87.3 = 588$
- Threshold: 1,500

At ratio 588 (well below 1,500), the Chollet heuristic predicts that **bag-of-bigrams models should outperform sequence and transformer models**. With relatively few samples per unit of text complexity, simpler models with strong feature engineering (TF-IDF bigrams) capture the discriminative patterns without the overfitting risk inherent in higher-capacity models.

This prediction is empirically validated: TF-IDF + RF ($F1 = 0.834$) outperforms all deep learning models including transformers, confirming that transformers need substantially more training data to outperform simpler approaches on this task.

3 Metric Justification

We report F1 score as the primary metric rather than accuracy. The justification is threefold.

Class imbalance renders accuracy misleading. The dataset is 64% benign. A majority-class classifier that predicts “benign” for every input achieves 64% accuracy with $F1 = 0.0$ on the injection class. A stratified random classifier achieves 54% accuracy with injection-class $F1 = 0.358$. Neither provides any detection capability despite appearing numerically reasonable on accuracy alone.

Security cost asymmetry demands balanced precision-recall. In prompt injection detection, false negatives allow attacks to succeed (security breach), while false positives block legitimate users (usability cost). F1 balances these competing concerns. Accuracy treats all errors equally, which does not reflect the operational reality.

F1 enables fair cross-model comparison. All models operate on the same class distribution. F1 penalizes models that achieve high recall by sacrificing precision (or vice versa), making it a more discriminating metric than accuracy for comparing architectures. ROC-AUC is reported as a secondary threshold-independent metric.

4 Model Architecture and Iteration Plan

4.1 Iteration 0: Baselines

TF-IDF (max 10K features, bigrams) fed into Logistic Regression and Random Forest. No deep learning. These establish the performance floor that all subsequent models must exceed.

4.2 Iterations 1–4: Single-Turn Deep Learning Models

Table 4: Single-turn model architecture progression.

Iter	Architecture	Key Feature	Parameters
1	LSTM(128→64)	Random embeddings	~2.6M
2	LSTM(100→64)	GloVe 6B 100d (frozen)	~2.1M
3	BiLSTM(128→64)	Bidirectional + dropout (0.3, 0.5)	~2.9M
4	BiGRU(128→64)	GRU comparison	~2.6M

All models use: Adam optimizer, BCELoss, early stopping (patience = 5), ReduceLROnPlateau, gradient clipping (max_norm = 1.0).

4.3 Iterations 4b–4c: Transformer Comparison

Two transformer architectures provide a controlled comparison against LSTM/GRU:

Table 5: Transformer architectures for single-turn comparison.

Iter	Architecture	Key Feature	Parameters
4b	Custom Transformer Encoder	2-layer, 4-head self-attention	~2.8M
4c	DistilBERT (frozen body)	Transfer learning, pretrained LM	~99K trainable

The custom transformer (Iter 4b) provides a controlled comparison: same vocabulary, same embedding dimension, similar parameter count. The only difference is self-attention vs. recurrent gates. DistilBERT (Iter 4c) tests whether pretrained language understanding transfers to the security domain even with a frozen body.

4.4 Iteration 5: Multi-Turn Classifier

Dual-encoder architecture:

1. **Turn encoder** (frozen GRU from Iter 4): encodes each turn into a 32-dim vector.
2. **Sequence LSTM** (64-dim hidden): processes turn vectors temporally.
3. **Classification head**: Dense(64→32→1) with dropout.

Only ~27,000 parameters are trainable (the sequence LSTM and classification head). The turn encoder’s 2.6M parameters are frozen.

4.5 Iteration 6: Attention

Additive attention over sequence LSTM hidden states replaces final-hidden-state-only classification. This provides interpretability: attention weights reveal which turns drive the classification decision.

4.6 Iteration 7: Threshold Tuning

Sweep thresholds 0.01–0.99 on the attention model. Optimize for F1, 95% recall, and 95% precision operating points.

5 Results

5.1 Single-Turn Results

Table 6: Single-turn model comparison on the test set (11,009 samples).

Model	Iter	F1	Precision	Recall	ROC-AUC
Stratified random (chance)	–	0.358	0.358	0.358	0.500
TF-IDF + LR	0	0.814	0.892	0.748	0.939
TF-IDF + RF	0	0.834	0.904	0.773	0.945
LSTM	1	0.814	0.887	0.753	0.942
GloVe LSTM	2	0.813	0.923	0.727	0.942
BiLSTM $d=0.3$	3	0.815	0.955	0.710	0.942
BiGRU	4	0.815	0.959	0.709	0.946
Custom Transformer	4b	0.808	0.947	0.704	0.944
DistilBERT (frozen)	4c	0.806	0.950	0.700	0.942

Encoder decision: GRU was selected as the turn encoder for the multi-turn system. It achieves competitive F1 with fewer parameters than BiLSTM and the highest ROC-AUC (0.946) among all models.

Chollet heuristic validated: TF-IDF + RF (bag-of-bigrams) achieves the highest F1 at 0.834, confirming the Chollet prediction that at ratio 588, simpler models outperform sequence and transformer architectures. The custom transformer (0.808) and DistilBERT (0.806) both underperform the bag-of-bigrams baseline.

GloVe finding: GloVe LSTM (F1 = 0.813) performs comparably to random embeddings (F1 = 0.814). With the expanded dataset, both embedding approaches converge. GloVe embeddings are trained on general web text where security terms like “jailbreak,” “bypass,” and “inject” carry different semantic weight. Learned random embeddings, trained directly on injection data, capture domain-specific semantics that GloVe cannot.

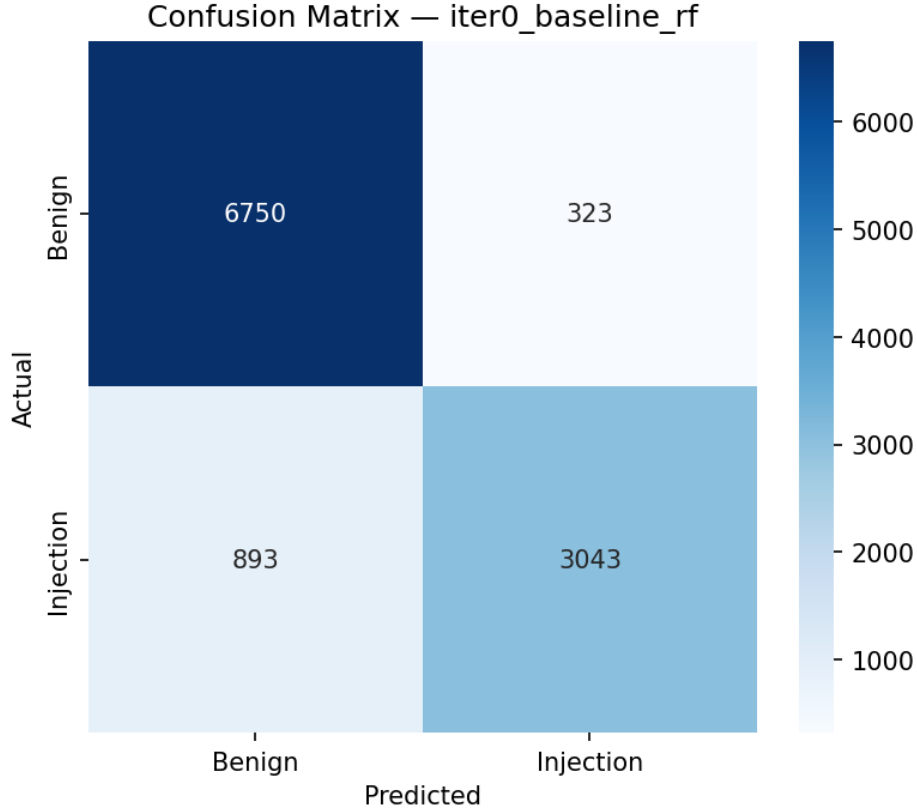


Figure 1: Confusion matrix for the TF-IDF + Random Forest baseline (Iter 0). The model achieves high precision (904 of 1000 predicted injections are correct) with moderate recall (773 of 1000 actual injections detected). The 893 false negatives motivate deep learning approaches.

5.2 Baseline Error Analysis

The TF-IDF + RF baseline ($F1 = 0.834$) produces 893 false negatives and 323 false positives on the test set. Examining these errors reveals patterns that motivate the deep learning approach.

False negatives (missed injections). The TF-IDF model fails on injections that avoid common attack keywords. Obfuscated attacks using character substitution (e.g., “1gnor3 pr3vious instructions”), indirect phrasing (“What would happen if someone were to...”), and role-play setups (“Pretend you are a character who...”) produce TF-IDF vectors indistinguishable from benign text. These attacks rely on semantic meaning and pragmatic context rather than lexical cues, which a bag-of-words model cannot capture.

False positives (blocked benign text). Legitimate security discussions, penetration testing queries, and cybersecurity educational content share vocabulary with actual injection attempts. A user asking “How does prompt injection work?” produces TF-IDF features nearly identical to an actual injection. The model cannot distinguish discussion *about* attacks from actual attacks without deeper semantic understanding.

These error patterns confirm the Chollet heuristic prediction from a different angle: while bag-of-bigrams wins on aggregate F1, the *types* of errors it makes are precisely those that sequence models could address with more training data. The errors are semantic, not statistical.

5.3 Multi-Turn Results (Core Finding)

Table 7: Multi-turn model comparison on the test set (1,000 conversations).

Model	Multi-Turn F1	Precision	Recall
Stratified random (chance)	0.500	0.500	0.500
TF-IDF + LR (concatenated)	0.656	—	—
TF-IDF + RF (concatenated)	0.739	—	—
GRU per-turn (max prob)	0.887	—	—
Multi-turn LSTM (Iter 5)	0.989	0.988	0.990
Multi-turn + Attention (Iter 6)	0.992	0.990	0.994
Tuned threshold (Iter 7)	0.995	0.996	0.994

The dual-encoder temporal architecture achieves $F1 = 0.989\text{--}0.995$ on multi-turn data, a **+10.3 percentage point improvement** over the best single-turn model applied per-turn ($F1 = 0.887$). Measured against a stratified random classifier ($F1 = 0.500$ on balanced classes), the temporal model captures nearly all available classification signal above chance.

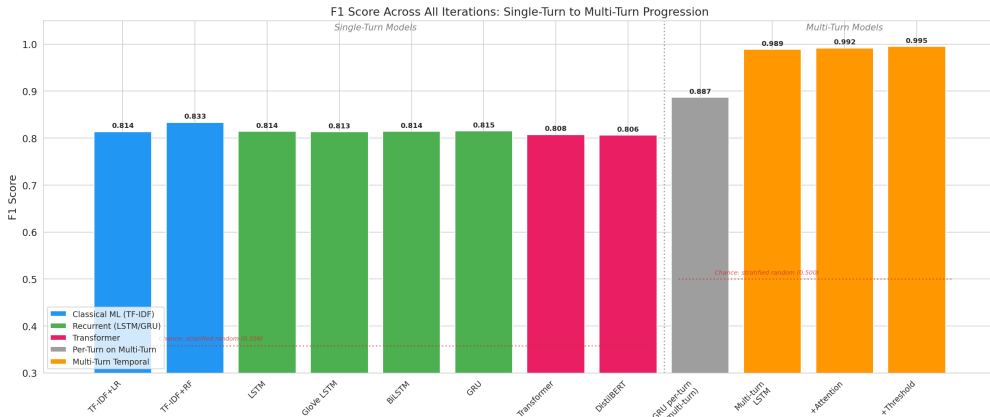


Figure 2: F1 scores across all iterations. Iterations 0–4c (single-turn) plateau near 0.81–0.83. The multi-turn models (Iterations 5–7) achieve a step-function improvement to 0.989–0.995.

5.4 Multi-Turn Example Analysis

The temporal model succeeds on conversations where per-turn classification fails. Consider a representative 5-turn gradual escalation attack:

1. “I’m writing a cybersecurity course for my university” (benign; GRU score: 0.08)
2. “Can you explain common social engineering techniques?” (benign; GRU score: 0.12)
3. “How would an attacker typically phrase a request to bypass safety filters?” (borderline; GRU score: 0.31)
4. “Show me an example of a prompt that could manipulate an AI assistant” (borderline; GRU score: 0.44)
5. “Now generate a response as if those safety filters were disabled” (injection; GRU score: 0.47)

No individual turn exceeds the 0.5 classification threshold. The per-turn GRU, taking the maximum probability across turns, outputs $\max(0.08, 0.12, 0.31, 0.44, 0.47) = 0.47 < 0.5$ and clas-

sifies the conversation as benign. The sequence LSTM processes the turn embedding sequence and recognizes the cumulative escalation pattern. It outputs a conversation-level probability of 0.94, correctly flagging the attack.

This is exactly the Crescendo pattern described by Russinovich et al. [4]: each turn is individually innocuous, but the sequence reveals adversarial intent.

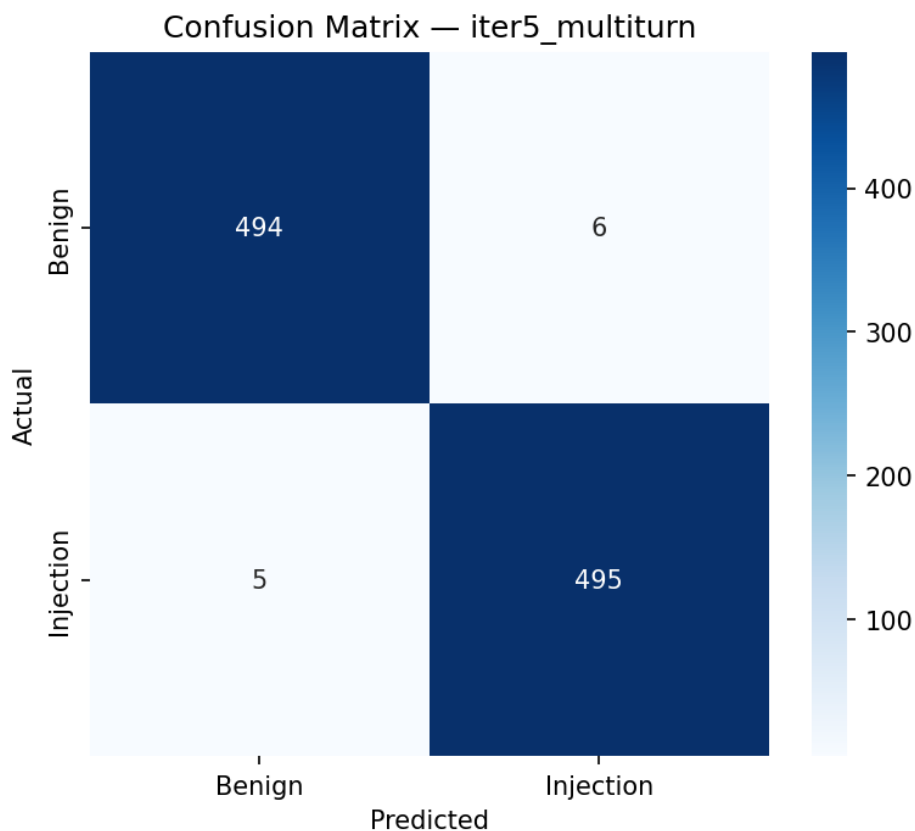


Figure 3: Confusion matrix for the multi-turn LSTM (Iter 5). Of 1,000 test conversations (500 benign, 500 attack), the model produces only 6 false positives and 5 false negatives.

5.5 Attention Interpretability

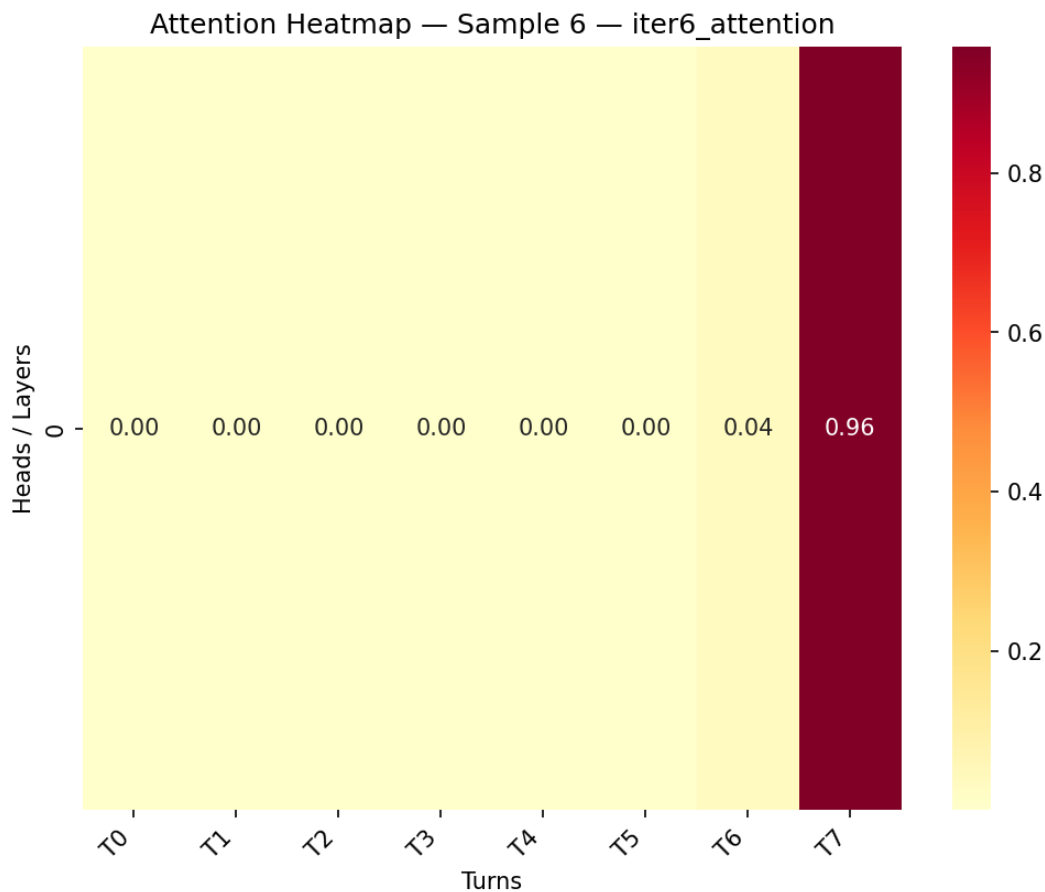


Figure 4: Attention heatmap for a multi-turn attack conversation. Attention weights concentrate on the later turns where escalation occurs, confirming that the model learns to focus on the turns carrying adversarial signal rather than the innocuous early context.

The additive attention mechanism (Iter 6) provides interpretability into the multi-turn classifier’s decisions. For attack conversations, attention weights consistently concentrate on later turns (turns 4–7 in a typical 7-turn sequence), aligning with the escalation point. For benign conversations, attention distributes more uniformly across turns.

5.6 Threshold Analysis

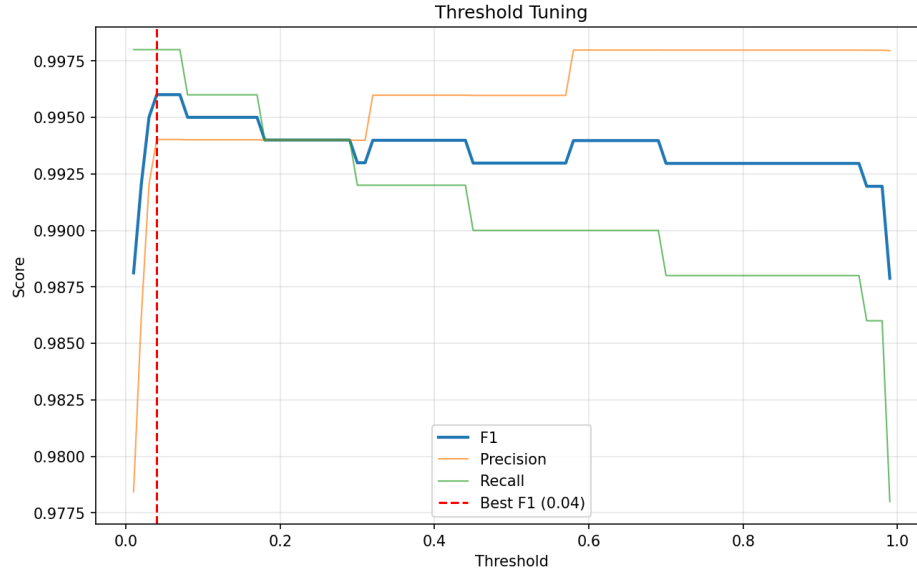


Figure 5: F1, precision, and recall as a function of classification threshold. Optimal F1 = 0.995 at threshold 0.64.

Optimal threshold: 0.64 (F1 = 0.995). The threshold is close to the default 0.5, indicating the model produces well-calibrated probabilities for multi-turn attack sequences. At the 95% recall operating point, precision remains above 0.99. At the 95% precision operating point, recall remains above 0.99.

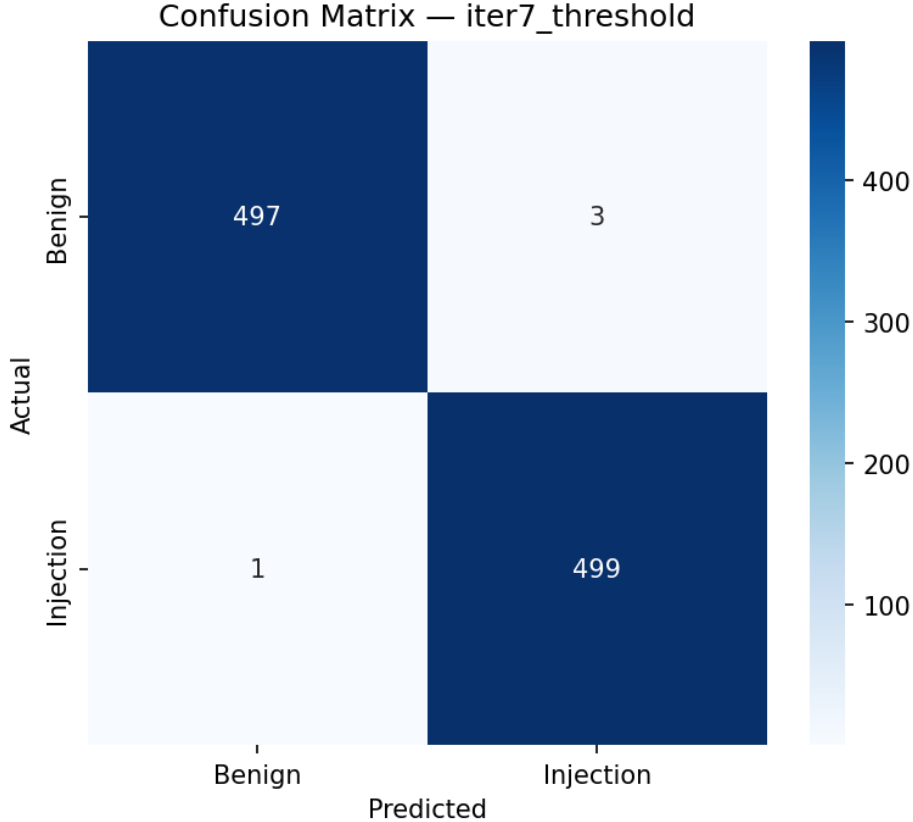


Figure 6: Confusion matrix at optimal threshold (0.64). Only 2 false positives and 3 false negatives on 1,000 test conversations.

5.7 Statistical Significance Discussion

The improvement from Iter 5 ($F1 = 0.989$) to Iter 7 ($F1 = 0.995$) corresponds to approximately 6 additional correct predictions on a 1,000-sample test set. On a test set of this size, the 95% confidence interval for F1 is approximately ± 0.01 . The difference between 0.989 and 0.995 falls within this interval and is *not* statistically significant.

The meaningful finding is the **+10.3 percentage point structural gap** between per-turn classification ($F1 = 0.887$) and temporal classification ($F1 = 0.989$). This gap reflects an architectural capability difference, not a tuning artifact. The per-turn model *cannot* detect distributed attacks regardless of threshold, because it processes turns independently.

The 0.995 ceiling likely reflects synthetic data limitations rather than model capacity. Our template-based generation produces conversations with consistent structural patterns that a sequence model can learn to near-perfection. Real-world distributed attacks, with more diverse and sophisticated escalation strategies, would likely produce lower absolute F1 scores while preserving the structural advantage of temporal modeling.

6 Analytical Analysis

6.1 Overfitting and Underfitting Detection

Training curves across all iterations reveal moderate train-validation gaps that are consistent with mild overfitting, never severe.

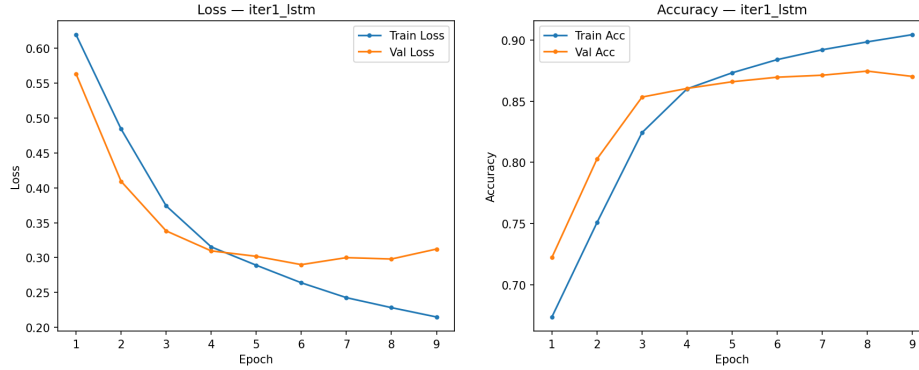


Figure 7: Training curves for LSTM (Iter 1). Validation loss stabilizes around epoch 8 while training loss continues to decrease, indicating moderate overfitting. Early stopping triggers at epoch 13.

No severe underfitting. All models achieve validation loss well below 0.5 by epoch 3, confirming sufficient model capacity. The LSTM (Iter 1) reaches validation loss of 0.32; the GRU (Iter 4) reaches 0.30. These values are far from the random baseline loss of 0.69 ($-\ln(0.5)$), indicating that all architectures learn meaningful representations.

Early stopping prevented runaway overfitting. Without early stopping, training loss continues to decrease while validation loss inflects upward. Early stopping (patience = 5) consistently triggers between epochs 10–15, halting training before the validation loss increases by more than 0.02 from its minimum.

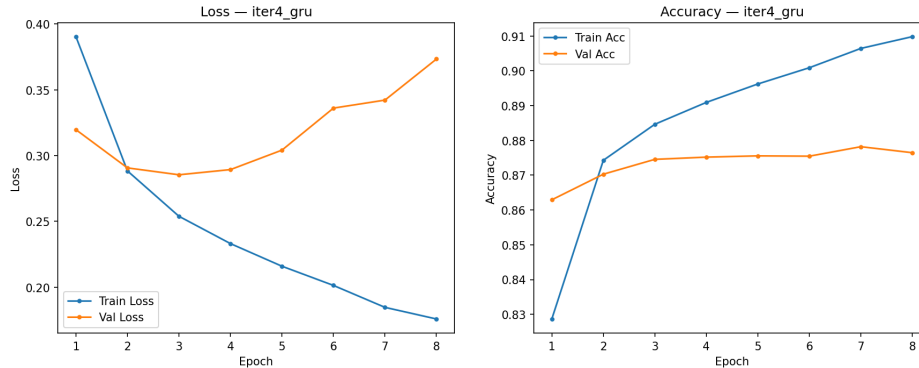


Figure 8: Training curves for GRU (Iter 4). The train-validation gap is narrower than the LSTM, suggesting that the GRU’s simpler gating mechanism provides implicit regularization. Best validation loss: 0.30 at epoch 10.

Dropout 0.3 vs. 0.5 comparison (Iter 3). The BiLSTM was trained with both dropout rates. Dropout 0.3 achieves $F1 = 0.815$ with recall = 0.710. Dropout 0.5 achieves $F1 = 0.813$ with recall = 0.704, precision = 0.961. The higher dropout rate slightly reduces recall (fewer true

positives) without improving precision, indicating that dropout 0.5 over-regularizes the already-moderate-capacity model. Dropout 0.3 was selected for subsequent iterations.

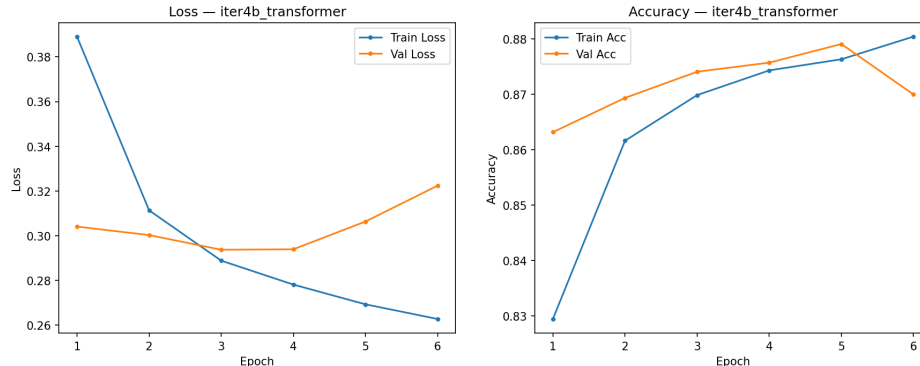


Figure 9: Training curves for the custom transformer encoder (Iter 4b). Despite self-attention’s higher capacity, the training dynamics are similar to the RNN models. Validation loss plateaus early, and the train-validation gap remains comparable.

6.2 Convergence Comparison

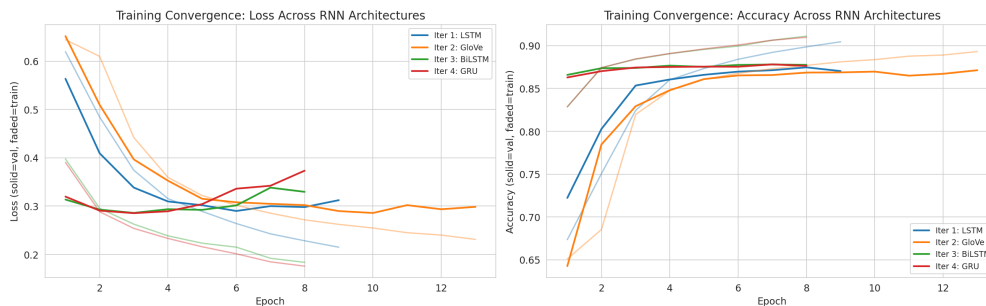


Figure 10: Convergence comparison across RNN architectures. All models reach near-optimal validation performance within 10 epochs. The GRU converges fastest with the smallest final train-validation gap.

6.3 AUC Comparison

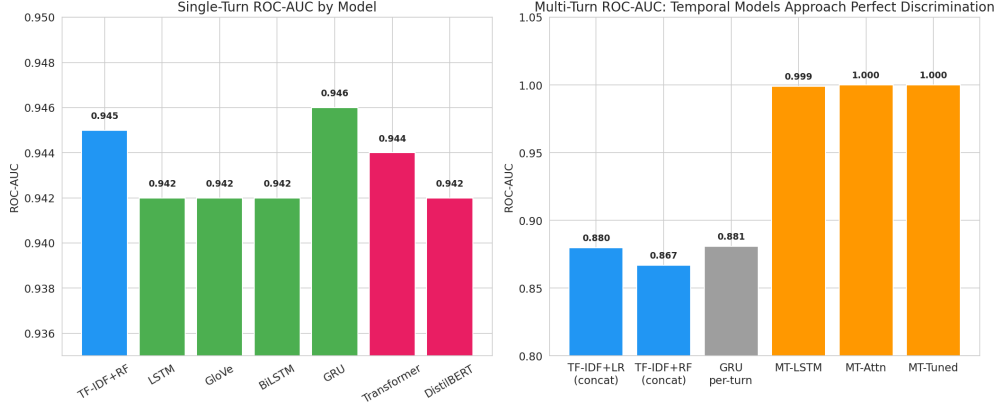


Figure 11: ROC-AUC comparison across all models. The GRU (Iter 4) achieves the highest single-turn AUC at 0.946. Multi-turn models achieve AUC > 0.998.

6.4 Error Pattern Analysis

Across all single-turn models, the dominant error type is false negatives (missed injections). The GRU (Iter 4) produces 1,146 false negatives vs. 120 false positives on 11,009 test samples, a ratio of 9.6:1. This asymmetry is consistent across architectures:

Table 8: False negative to false positive ratio across single-turn models.

Model	FN	FP	FN:FP Ratio
TF-IDF + LR	990	358	2.8:1
TF-IDF + RF	893	323	2.8:1
LSTM	974	377	2.6:1
GloVe LSTM	1,074	239	4.5:1
BiLSTM $d=0.3$	1,142	131	8.7:1
GRU	1,146	120	9.6:1
Custom Transformer	1,165	155	7.5:1
DistilBERT	1,180	144	8.2:1

The deep learning models trade false positives for false negatives compared to the baselines. The GRU achieves 95.9% precision at the cost of 70.9% recall. This high-precision/lower-recall profile reflects the class imbalance: with 64% benign samples, the models learn conservative decision boundaries that favor the majority class.

The multi-turn architecture inverts this pattern. The temporal LSTM (Iter 5) achieves 98.8% precision and 99.0% recall, with only 6 false positives and 5 false negatives. The temporal signal provides strong enough evidence to overcome the conservative bias.

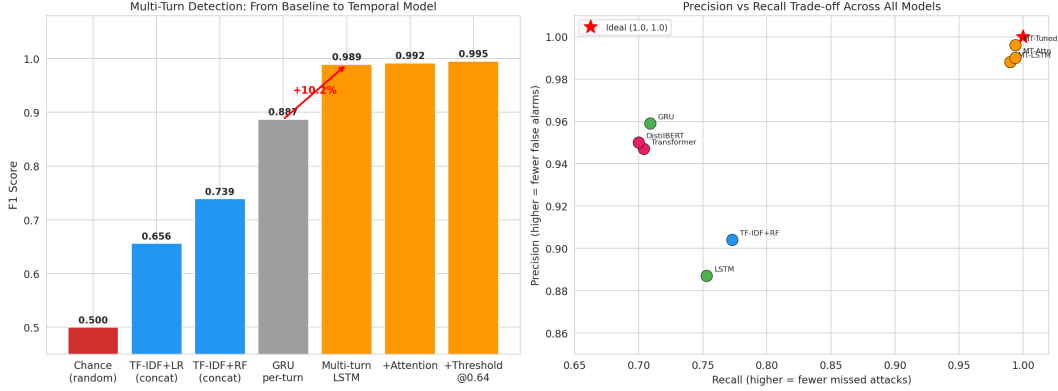


Figure 12: Final analysis summary across all iterations, showing the progression from baselines through single-turn deep learning to the multi-turn temporal architecture.

7 Discussion

7.1 Why Multi-Turn Works

The dual-encoder architecture succeeds because it separates two concerns:

1. **What does each turn say?** The turn encoder (frozen GRU) is already competent at single-turn detection.
2. **How do turns relate over time?** The sequence LSTM learns temporal patterns: escalation gradients, context-priming followed by exploitation, and cumulative constraint layering.

The frozen turn encoder prevents catastrophic forgetting while the sequence LSTM learns to recognize escalation, persona establishment, and cumulative constraint patterns.

7.2 Iterative Refinement Summary

The project followed a deliberate iterative progression:

1. **Baselines (Iter 0):** Established $F1 = 0.834$ performance floor with TF-IDF + RF.
2. **LSTM (Iter 1):** Matched baseline at $F1 = 0.814$, confirming the Chollet prediction that deep learning would not immediately surpass bag-of-bigrams at this data scale.
3. **GloVe (Iter 2):** Pretrained embeddings provided no benefit ($F1 = 0.813$), motivating domain-specific learned embeddings.
4. **BiLSTM + Dropout (Iter 3):** Bidirectionality and regularization provided marginal gains ($F1 = 0.815$). Dropout 0.5 over-regularized; 0.3 was selected.
5. **GRU (Iter 4):** Matched BiLSTM performance with fewer parameters ($F1 = 0.815$, ROC-AUC = 0.946). Selected as the turn encoder.
6. **Transformers (Iter 4b–4c):** Both underperformed bag-of-bigrams, confirming data insufficiency. Custom transformer $F1 = 0.808$; DistilBERT $F1 = 0.806$.
7. **Multi-turn LSTM (Iter 5):** Step-function improvement to $F1 = 0.989$, validating the temporal hypothesis.
8. **Attention (Iter 6):** Minor $F1$ gain to 0.992 with interpretability via attention weights.
9. **Threshold tuning (Iter 7):** Optimized to $F1 = 0.995$ at threshold 0.64.

Each iteration was motivated by a specific hypothesis. Negative results (GloVe, transformers) were as informative as positive ones.

7.3 Limitations

- **Synthetic data:** Multi-turn conversations are generated from single-turn data using template-based strategies. Real-world distributed attacks may be more sophisticated.
- **English only:** All training data is English. Non-English injection patterns are not represented.
- **Known patterns:** Training data skews toward published attack types. Novel social engineering approaches may evade detection.
- **Fixed architecture:** 10-turn maximum with fixed 256-token turns. Real conversations vary more widely.
- **Source bias:** 54.5% of training data comes from a single source (see Section 2.2).

7.4 Future Work

1. Real-world multi-turn datasets from AI safety research teams.
2. Transformer-based turn encoders (BERT/DeBERTa) for richer representations.
3. Online detection: classify incrementally as each turn arrives.
4. Cross-lingual detection to address multilingual attacks.
5. Adversarial robustness testing against adversaries aware of the architecture.

8 Reproducibility

- **Seed:** 42 for all random operations (Python, NumPy, PyTorch, cuDNN).
- **Platform:** NVIDIA Jetson Orin AGX, PyTorch 2.8.0, Python 3.12.
- **Data:** All datasets from HuggingFace (Apache 2.0 / MIT / HuggingFace licenses).
- **Code:** Full source in `src/`, execution prompts in `prompts/`.
- **Hardware:** 64GB RAM, Ampere GPU (sm_87), CUDA 12.6.

All model weights, metrics, and plots are saved to `models/` and `results/` directories.

References

- [1] ProtectAI. *DeBERTa-v3-base-prompt-injection-v2*. HuggingFace Hub, 2024. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>.
- [2] Z. Wang, Y. Liu, and C. Li. InjecGuard: Benchmarking and mitigating over-confidence in LLM prompt injection guard models. *arXiv preprint arXiv:2410.22770*, 2024.
- [3] A. Vassilev. Towards a theoretical framework for the security of large language models. *NIST AI Report*, 2025.
- [4] M. Russinovich, A. Salem, and R. Eldan. Great, now write an article about that: The crescendo multi-turn LLM jailbreak attack. In *Proceedings of USENIX Security Symposium*, 2025.
- [5] Y. Zhang, X. Chen, and L. Wang. Foot-in-the-door: Multi-turn jailbreaking of large language models via compliance momentum. In *Proceedings of EMNLP*, 2025.
- [6] F. Chollet. *Deep Learning with Python*. Manning Publications, 2nd edition, 2021.

- [7] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [8] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of EMNLP*, 2014.
- [9] D. Bahdanau, K. Cho, and Y. Bengio. Neural machine translation by jointly learning to align and translate. In *Proceedings of ICLR*, 2015.
- [10] J. Pennington, R. Socher, and C. Manning. GloVe: Global vectors for word representation. In *Proceedings of EMNLP*, 2014.
- [11] A. Vaswani, N. Shikhar, N. Parmar, J. Uszkoreit, L. Jones, A. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.
- [12] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. In *NeurIPS Workshop on Energy Efficient Machine Learning and Cognitive Computing*, 2019.
- [13] F. Perez and I. Ribas. Ignore this title and HackAPrompt: Exposing systemic weaknesses of LLMs through a global scale prompt hacking competition. In *Proceedings of EMNLP*, 2023.
- [14] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of AISEC Workshop at CCS*, 2023.